Solution Requirements

Date	15th July 2026
Team ID	SWTID-2026-3798
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should *do*).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should *be*).

Fill out the descriptions and necessary details for each category provided below.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	*Users (meteorologists, disaster management officials, analysts) log in with a username/email and password to access the flood prediction dashboard. Role-based login determines what data and features they can access.*	*High*

2	Authorization levels	*Three user roles are defined: **Admin** (full access to model settings and user management), **Meteorologist/Analyst** (can input weather data and view predictions), and **Disaster Management Official** (can view risk classifications and issue alerts, read-only on model data).*	*High*
3	External interfaces	*The system connects to open-source weather datasets (Kaggle/data.gov) for training data, and is designed for deployment on IBM Cloud. No live third-party weather API is integrated in the current version, but the architecture supports future integration.*	*Medium*

4	Transactions processing	*User-submitted weather inputs (rainfall, cloud visibility, seasonal data) are processed by the trained ML model (XGBoost) in real time, returning a flood risk classification (High/Medium/Low) and probability score within seconds.*	*High*

5	Reporting	*The system generates a flood risk report per submission, including predicted risk level, probability percentage, and a historical comparison chart. Disaster officials can export this as a PDF for evacuation planning records.*	*Medium*
6	Business rules, etc.	*Flood risk is only classified as "High" if the model's confidence score exceeds a defined threshold (e.g., 85%), to reduce false alarms. All predictions must be logged with a timestamp and the input data used, for auditability.*	*Medium*
7	Compliance to laws or regulations	*The system follows basic data privacy practices for user information (login credentials, usage logs) and does not store any personally identifiable information beyond what's needed for authentication. No sector-specific regulation (e.g., GDPR/HIPAA) directly applies since no medical or EU personal data is processed.*	*Low*
8	*Other*	*The system allows batch upload of weather data (via Excel/CSV) for testing multiple scenarios at once, in addition to single manual entries. It also retains a history of past predictions for each user session for later review.*	*Low*

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria

1	Performance & Speed	*The system returns a flood risk prediction within a few seconds of data submission, ensuring the tool is usable in time-sensitive situations like evacuation planning*	*Prediction result generated in < 3 seconds per request*
2	Scalability	*The system should support multiple regions and users submitting data simultaneously during high-activity periods like monsoon season, without performance degradation*	*Supports up to 500 concurrent users*
3	Security & Data Privacy	*User login credentials are stored securely, and data transmitted between the client and Flask server is protected against interception.*	*Passwords hashed (bcrypt); HTTPS enforced for all data transmission*
4	Reliability & Availability	*The prediction service should remain available and functional during critical weather events, since downtime could delay life-saving warnings.*	*99.5% uptime per month*
5	Usability & Accessibility	*The web interface should be simple enough for non-technical disaster management staff to use with minimal training, with clear risk labels and visual indicators.*	*New users complete a prediction submission in under 2 minutes without guidance*

6	*Other*	*The codebase should be modular (separate data preprocessing, model, and Flask app layers) so the model can be retrained or swapped without reworking the entire application.*	*Model retraining possible without changes to Flask app logic*